Shyamoli Textile Engineering College
Department of Computer Science and Engineering

CSE-4237: Digital Image Processing

Lec-14: Color Image Processing

Digital Image Processing by Rafael C. Gonzalez and Richard Eugene Woods (Chapter-6)

Md. Ariful Islam
Assistant Professor
Dept. of Robotics and Mechatronics Engineering
University of Dhaka

2.6 Pseudocolor Image Processing

Pseudocolor (also called false color) image processing *consists of assigning colors to gray values* based on a *specified criterion*.

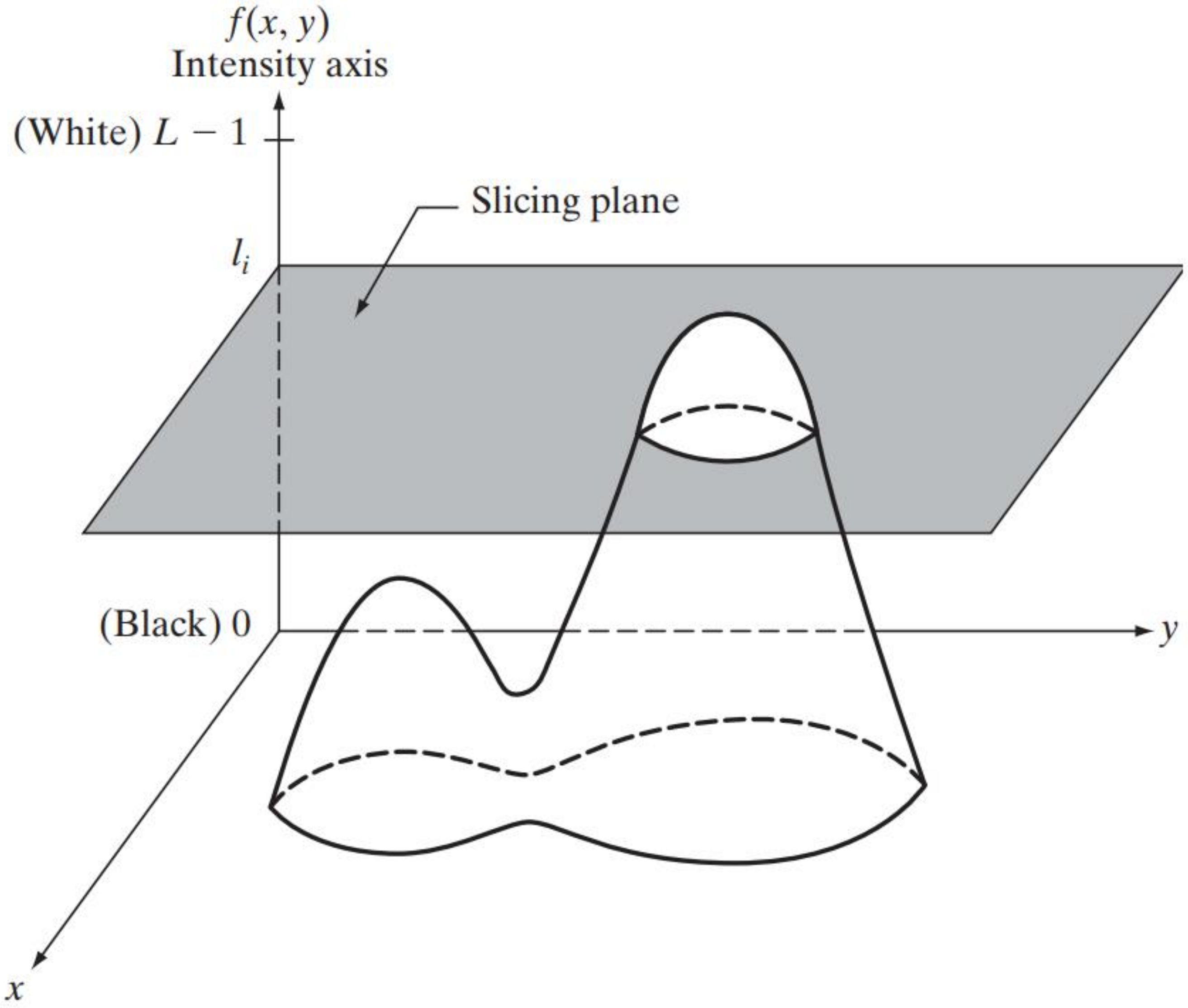
The term *pseudo or false color* is used to differentiate the process of *assigning colors to monochrome images* from the processes associated with true color images.

2.6.1 Intensity Slicing

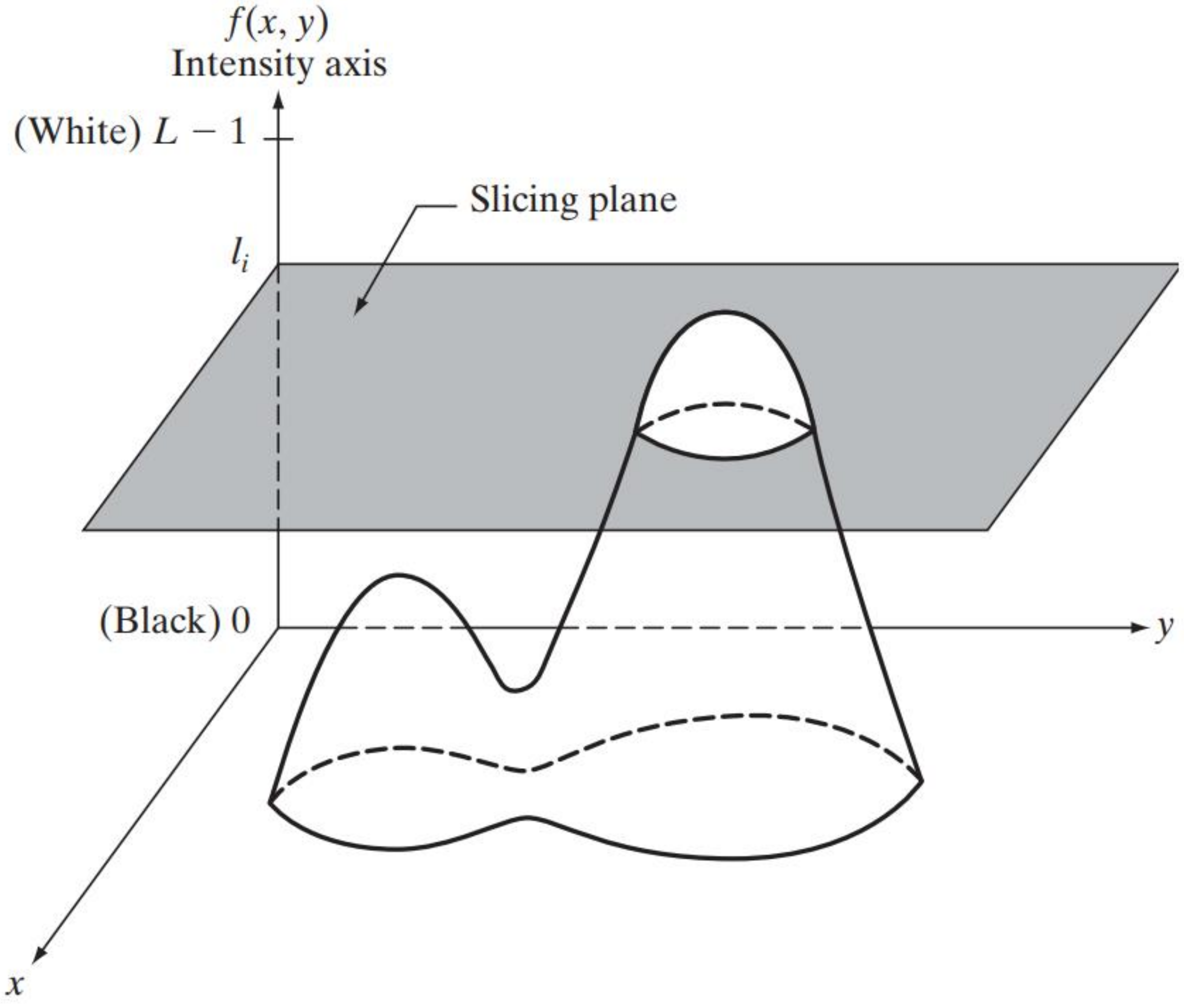
The technique of intensity (sometimes called density) slicing and color coding is one of the simplest examples of pseudocolor image processing.

If an image is interpreted as a 3-D function, the method can be viewed as *one of placing planes parallel to the coordinate plane* of the image; *each plane then "slices" the function in the area of intersection*.

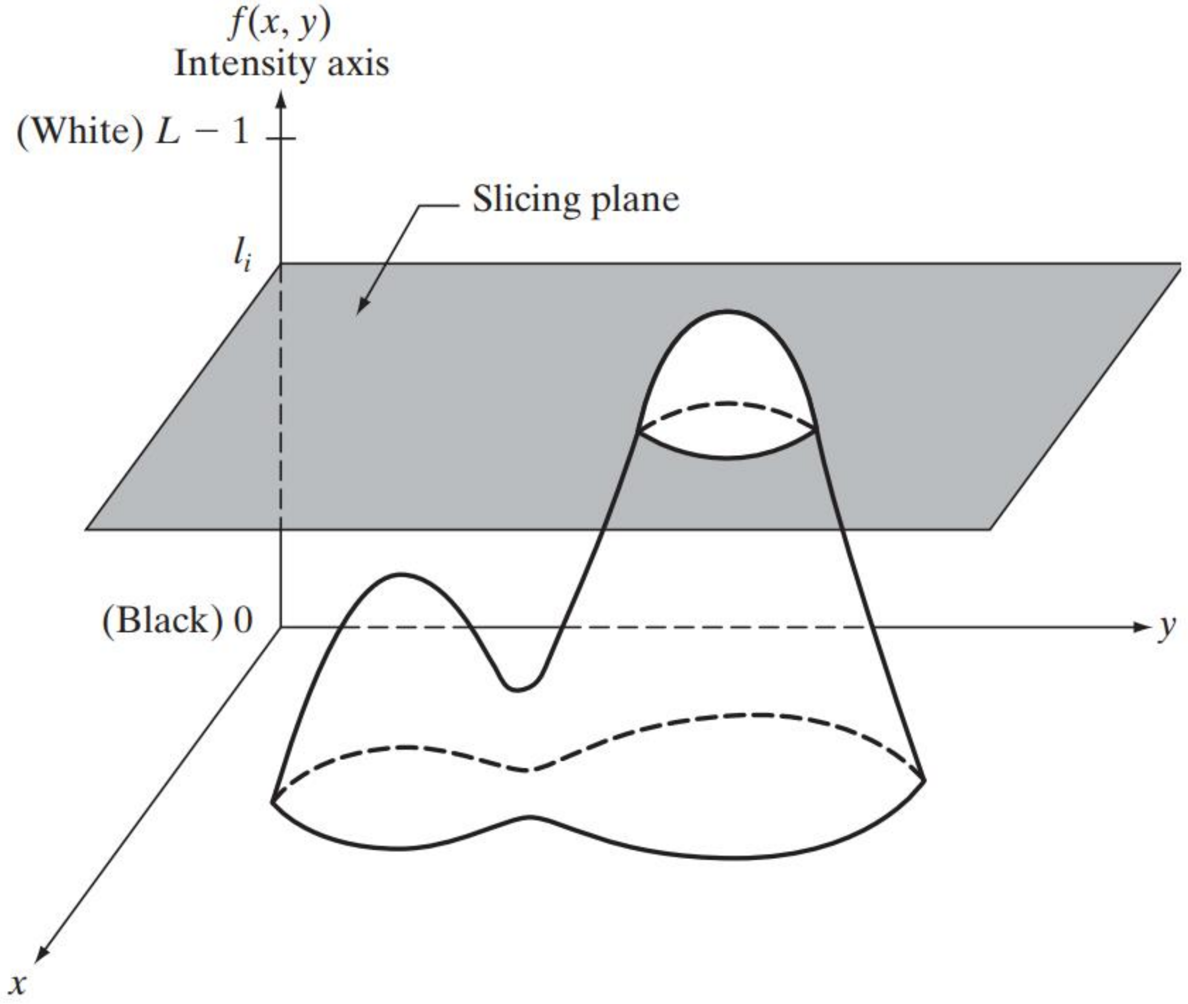
Figure shows an example of using a plane at $f(x,y) = l_i$ to <u>*slice the image function into two levels.*</u>



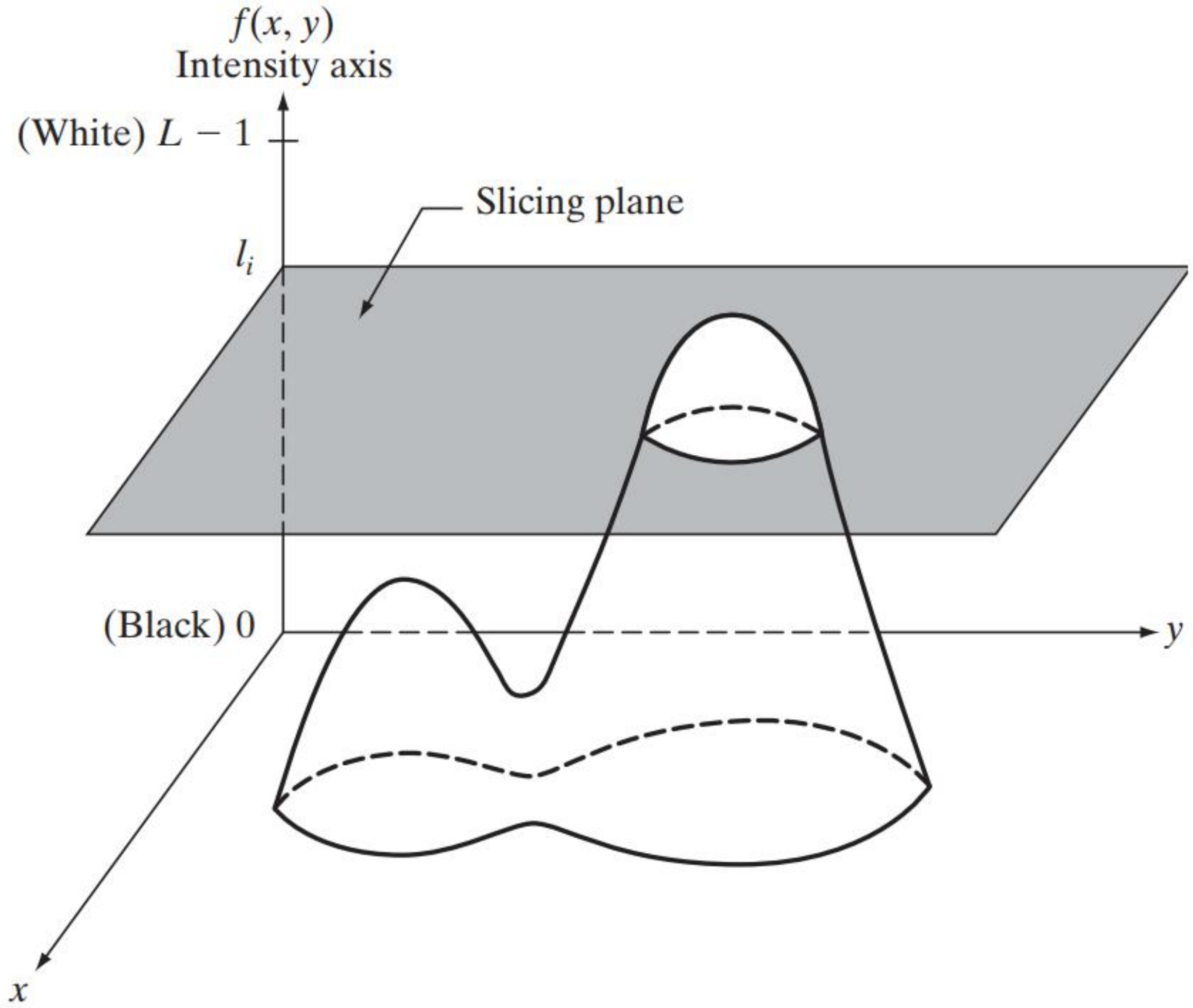
If a *different color is assigned to each side* of the plane shown in Fig., any pixel whose *intensity level is above the plane will be coded with one color*, and any pixel *below the plane will be coded with the other*.



Levels that lie on the plane itself may be arbitrarily assigned one of the two colors. The result is a two-color image whose *relative appearance can be controlled by moving the slicing plane up and down* the intensity axis.



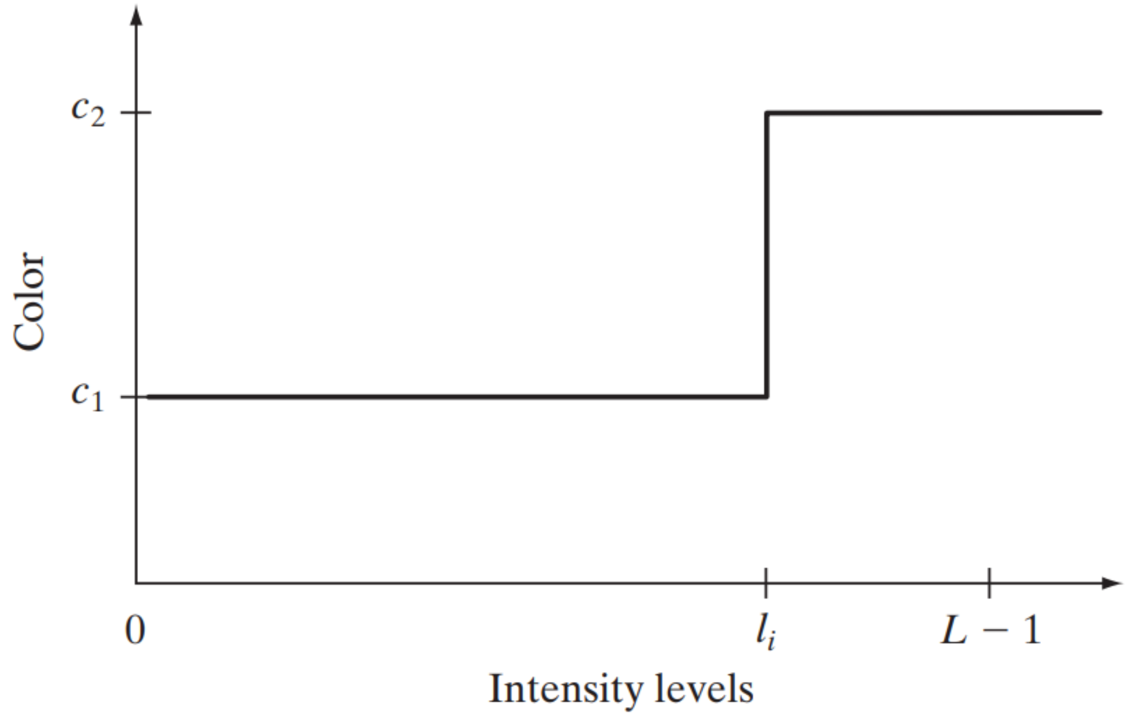
In general, the technique may be summarized as follows. Let $[0, L-1]$ represent the gray scale, let level l_0 represent black $[f(x, y) = 0]$, and level l_{L-1} represent white $[f(x, y) = L-1]$.



An alternative representation of the intensity slicing technique

where c_k is the color associated with the kth intensity interval V_k defined by the partitioning planes at $l = k - 1$ and $l = k$.

$$f(x, y) = c_k \qquad \text{if } f(x, y) \in V_k$$



According to the mapping function, <u>*any input intensity level is assigned one of two colors, depending on whether it is above or below the value of l_i*</u>

When <u>*more levels are used, the mapping function takes on a staircase form.*</u>

Let $I(x,y)$ represent the intensity function of the image, where x and y are spatial coordinates, and $I(x,y)$ is the intensity at that point.

Define Intensity Range:

Define a set of intensity intervals $[I_1,I_2],[I_3,I_4],\ldots,[I_k,I_{k+1}]$, where each interval corresponds to a different color range.

Assign Color:

For each pixel with intensity value $I(x,y)$, assign a color based on which interval the intensity value falls into.

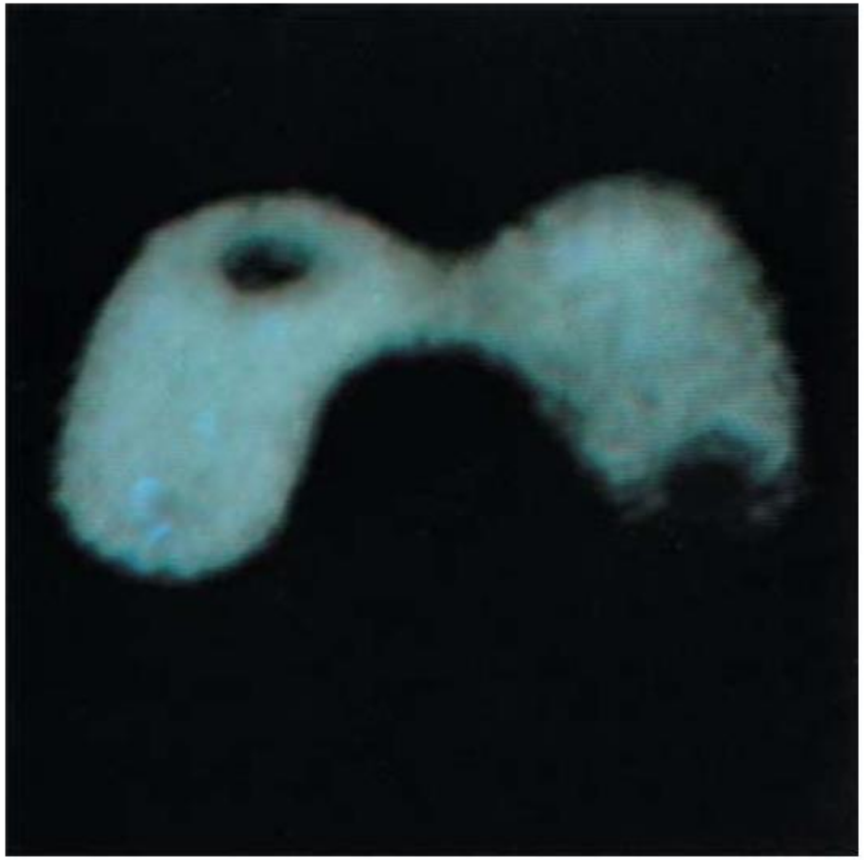
Mathematically, this can be expressed as:

$$C(x,y) = \begin{cases} \text{Color}_1 & \text{if } I_1 \le I(x,y) < I_2 \\ \text{Color}_2 & \text{if } I_3 \le I(x,y) < I_4 \\ \dots \\ \text{Color}_k & \text{if } I_k \le I(x,y) < I_{k+1} \end{cases}$$

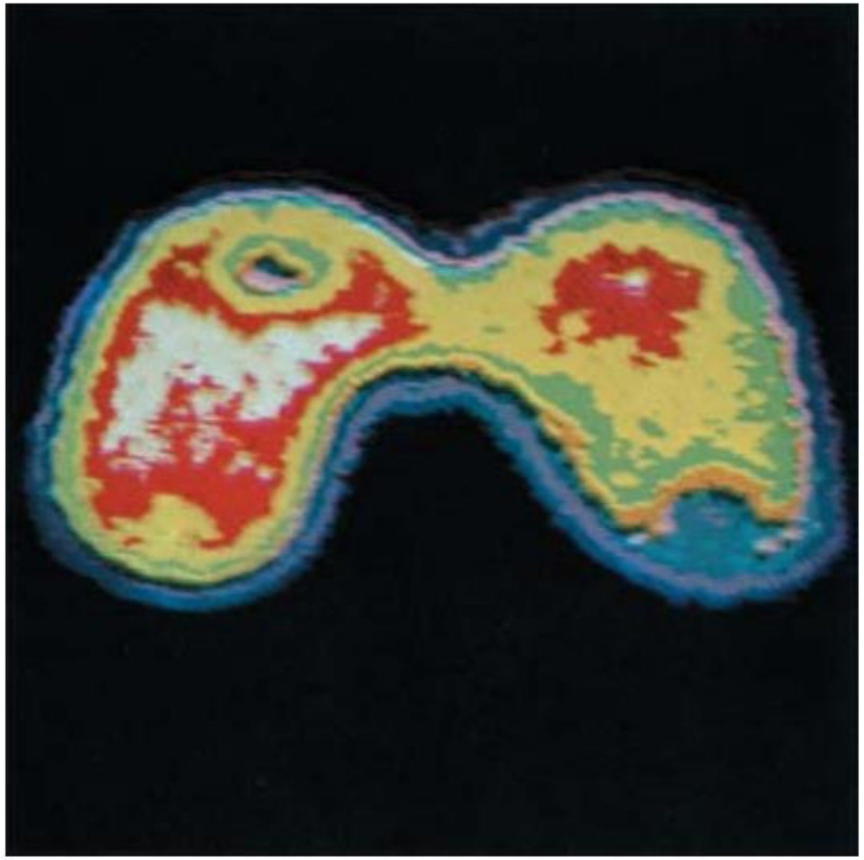
Here, $C(x,y)$ represents the color assigned to the pixel at coordinates (x,y) based on its intensity $I(x,y)$.

$$C(x,y) = \begin{cases} \text{Blue} & \text{if } 50 \le I(x,y) < 100 \\ \text{Green} & \text{if } 100 \le I(x,y) < 150 \\ \text{Red} & \text{if } 200 \le I(x,y) \le 255 \end{cases}$$

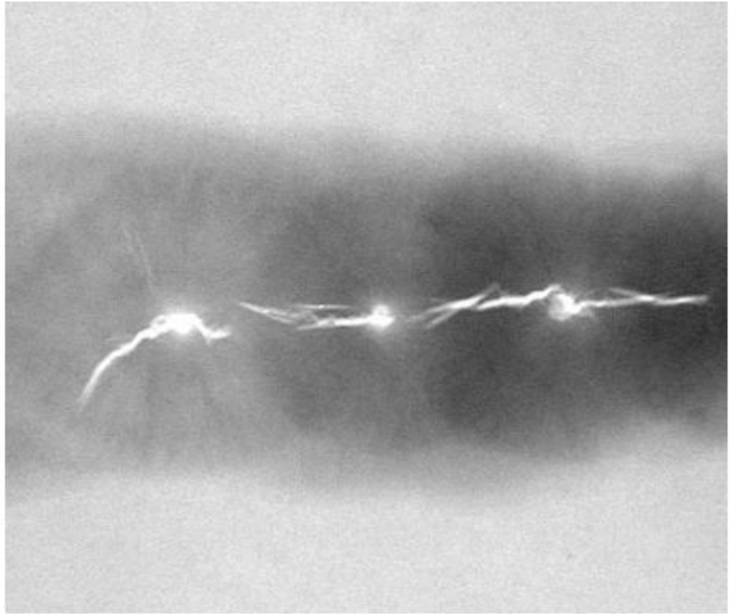
Example: Figure (a) is a monochrome image of the Picker Thyroid Phantom (a radiation test pattern), and Fig. (b) is the result of intensity slicing this image into eight color regions.



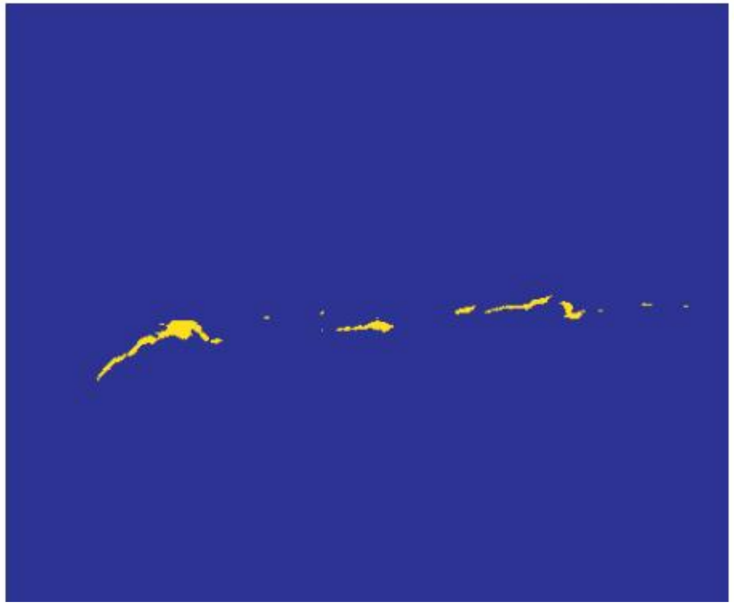
(a)



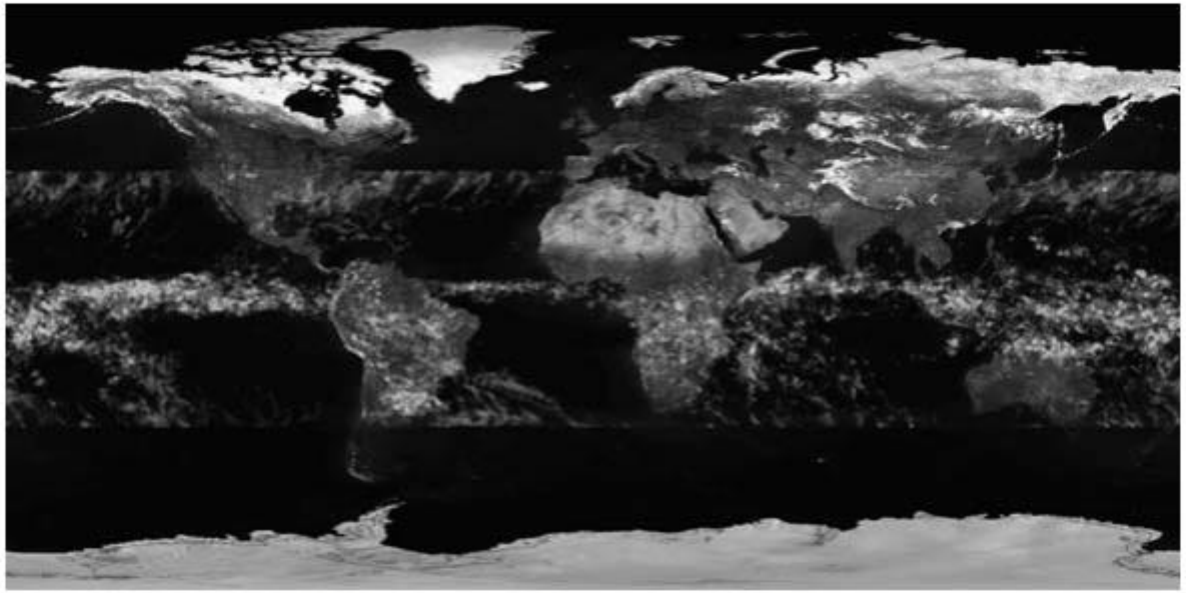
(b)



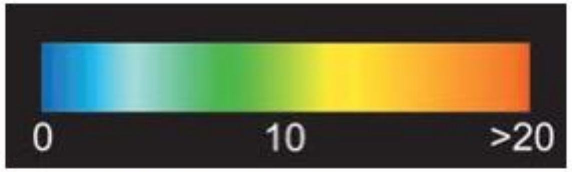
An X-ray image of a weld (the horizontal dark region) containing several cracks and porosities (the bright, white streaks running horizontally through the middle of the image)



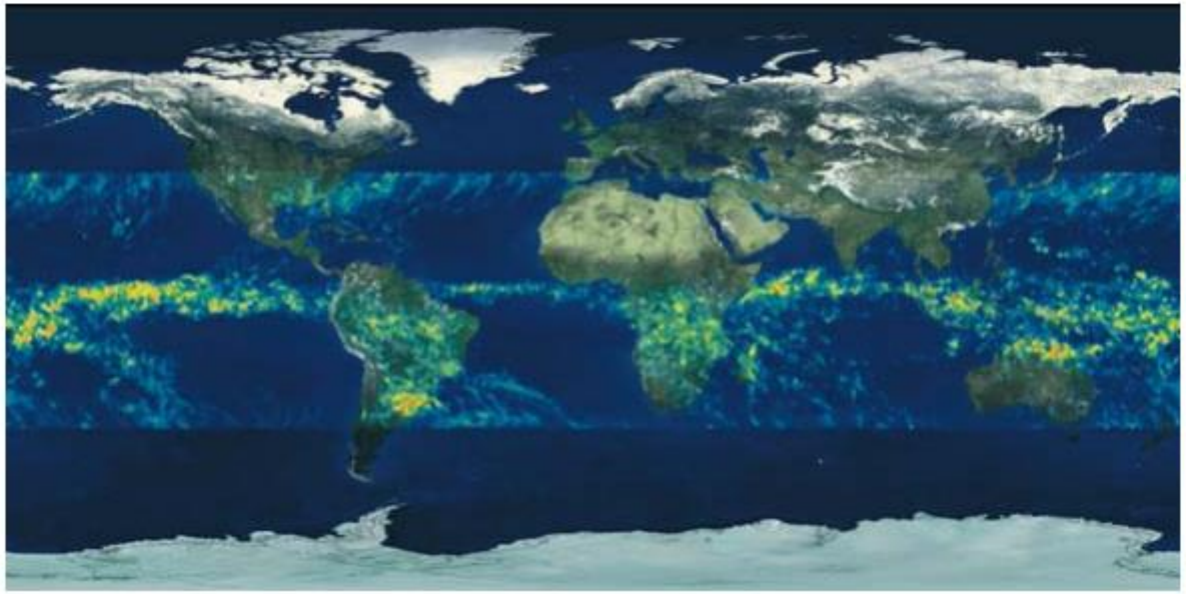
human error rates would be lower if images were displayed in this form

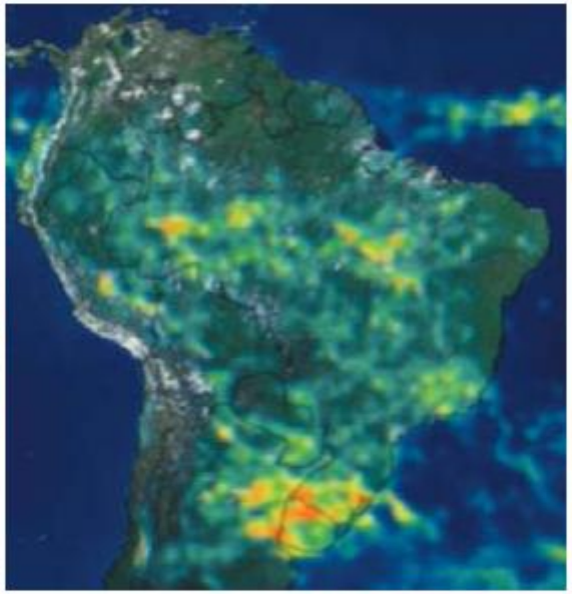


(a) Gray-scale image in which intensity (in the lighter horizontal band shown) corresponds to average monthly rainfall.



(b) Colors assigned to intensity values.



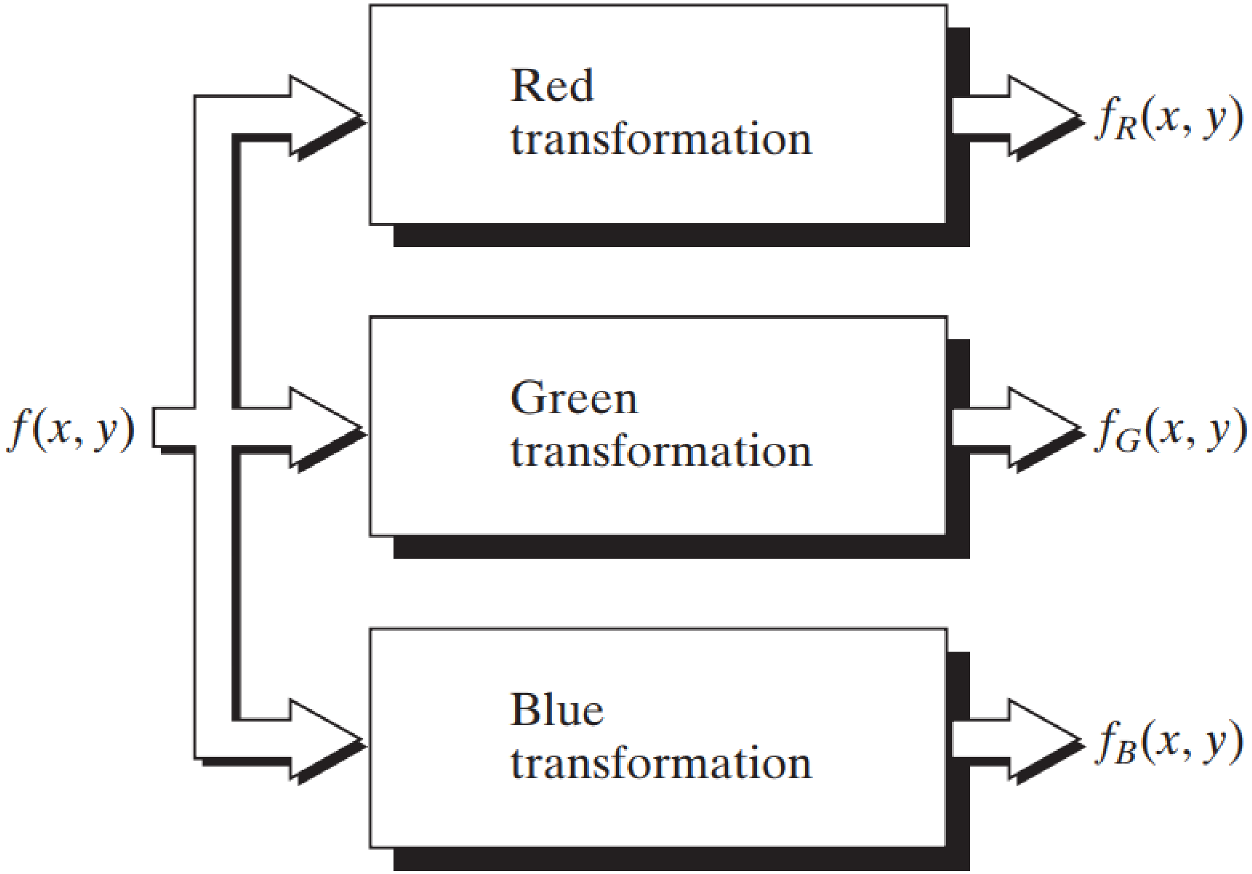


(c) Color-coded image

(d) Zoom of the South American region.

2.6.2 Intensity to Color Transformations

Other types of transformations are more general and thus are *capable of achieving a wider range of pseudocolor enhancement* results than the simple slicing technique.



Functional block diagram for pseudocolor image processing. f_R, f_G, and f_B are fed into the corresponding red, green, and blue inputs of an RGB color monitor.

Basically, the idea underlying this approach is to *perform three independent transformations on the intensity of any input pixel*. The three results are *then fed separately into the red, green, and blue channels* of a color television monitor.

****** This method produces a composite image* whose *color content is modulated by the nature of the transformation functions*.

- Note that these are transformations on the intensity values of an image and *are not functions of position*.

Mathematical Representation of Intensity to Color Transformations:

Let $I(x,y)$ be the intensity value at position (x,y) in the image. The *transformation functions for the Red, Green, and Blue* channels can be represented as *independent functions of the intensity*:

$$R(x,y) = f_R(I(x,y))$$

$$G(x,y) = f_G(I(x,y))$$

$$B(x,y) = f_B(I(x,y))$$

R(x,y), $G(x,y)$, and $B(x,y)$ represent the intensity values for the red, green, and blue channels respectively at pixel (x,y).

f_R , f_G , and f_B are *transformation functions that map the original intensity* value $I(x,y)$ to new intensity values for each channel.

Common Transformation Functions

Linear Transformation

A common transformation is a linear function where the intensity is scaled by a constant. For example, for the red channel, we could have:

$$R(x, y) = \alpha_R \cdot I(x, y)$$

where α_R is a scaling factor (e.g., $\alpha_R = 1.5$).

Logarithmic Transformation

Sometimes, intensity values are transformed using a logarithmic function to compress the dynamic range of pixel intensities.

$$R(x, y) = \log(\beta_R \cdot I(x, y) + 1)$$

where β_R is a constant that determines the scale.

Sigmoidal Transformation

A sigmoidal function can be used for non-linear enhancement of intensity ranges, producing a more visually appealing mapping. For example:

$$G(x,y) = \frac{1}{1 + e^{-k_G \cdot (I(x,y) - I_0)}}$$

where k_G controls the steepness and I_0 is the intensity value around which the transformation is centered.

Exponential Transformation

In some cases, an exponential transformation might be applied to emphasize high-intensity values:

$$B(x, y) = e^{\gamma_B \cdot I(x,y)}$$

where γ_B is a constant.

Constructing the Final Image

Once the three transformation functions are applied to the intensity values, they produce three output values corresponding to the red, green, and blue channels. The final output image is composed of these three transformed intensity values:

$$\mathrm{RGB}(x, y) = (R(x, y), G(x, y), B(x, y))$$

$$R(x, y) = 1.2 \cdot I(x, y)$$

$$G(x, y) = \log(1 + I(x, y))$$

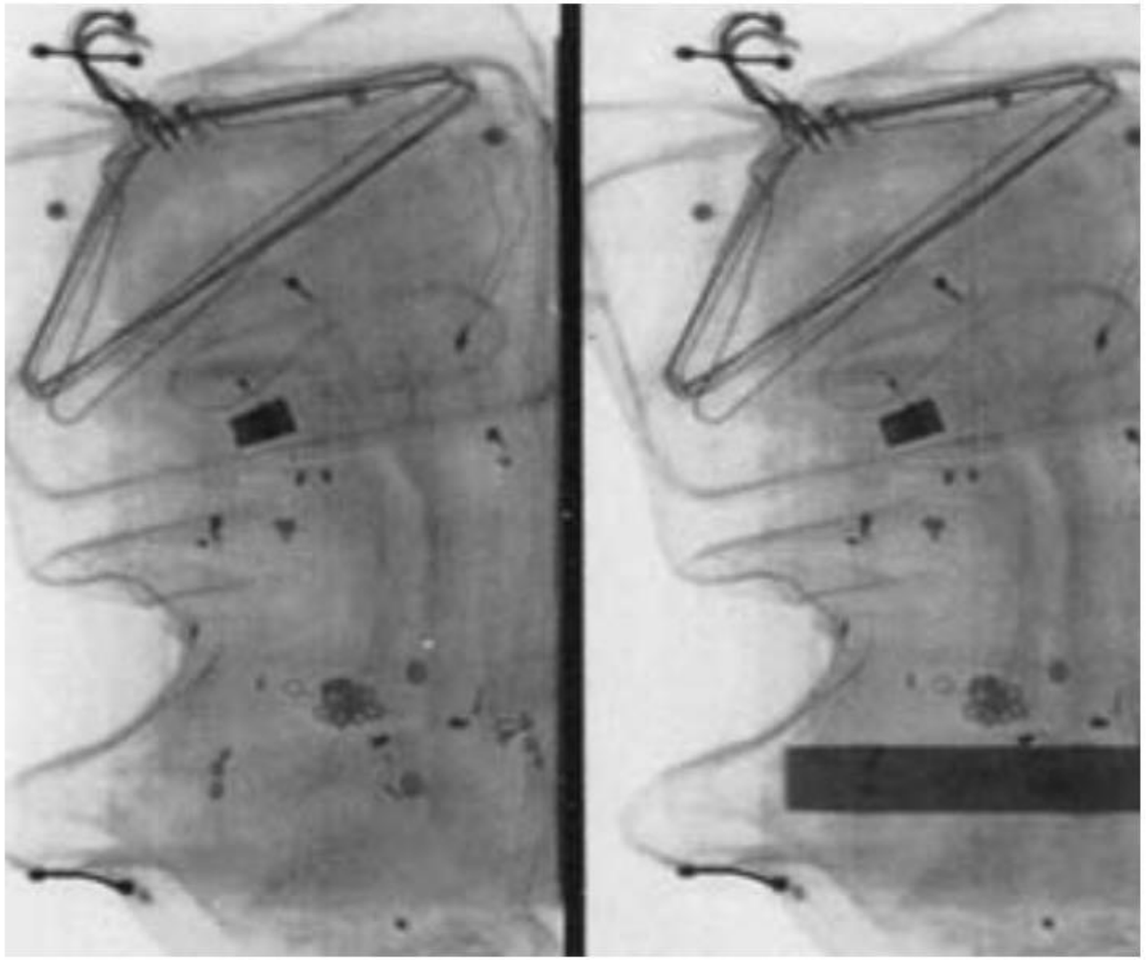
$$B(x, y) = e^{0.01 \cdot I(x, y)}$$

Example: Figure (a) shows two *monochrome images of luggage* obtained from an *airport X-ray scanning system*.

i. The image on the left contains ordinary articles.

ii. The image on the right contains the same articles, as well as a *block of simulated plastic explosives*.

- The purpose of this example is to illustrate the *use of intensity level to color transformations* to obtain various degrees of enhancement.

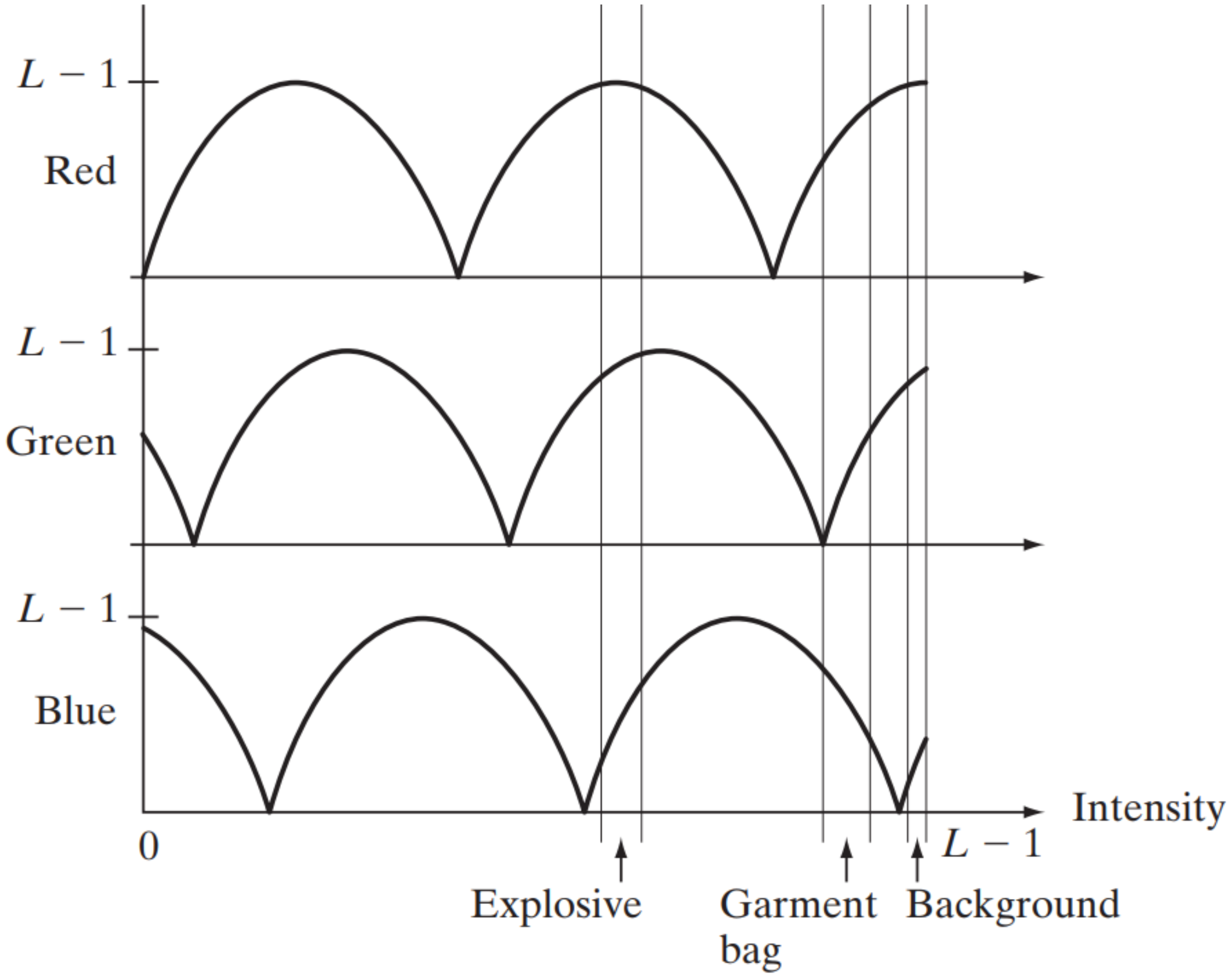


(a)

Figure (b) shows the *<u>transformation functions</u>* used. These sinusoidal functions *<u>contain regions of relatively constant value</u>* around the **peaks as well as regions that change rapidly** near the valleys.

<u>Changing the phase and frequency</u> of each sinusoid can *<u>emphasize (in color) ranges in the gray scale</u>*.

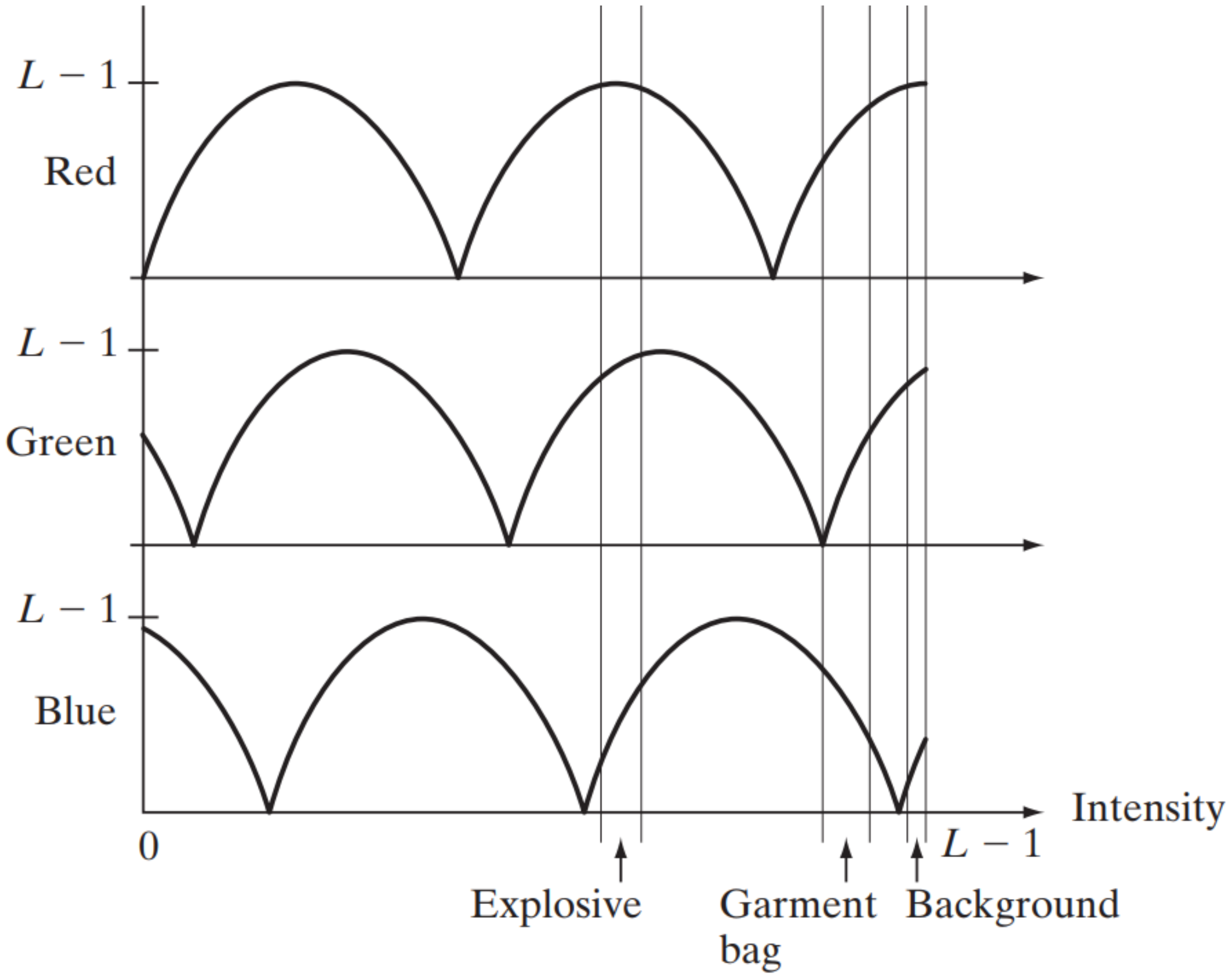
*** For instance, *<u>if all three transformations have the same phase and frequency, the output image will be monochrome</u>*.



(b)

A small change in the phase between the three transformations *produces little change in pixels* whose intensities correspond to peaks in the sinusoids, especially if the sinusoids have broad profiles (low frequencies).

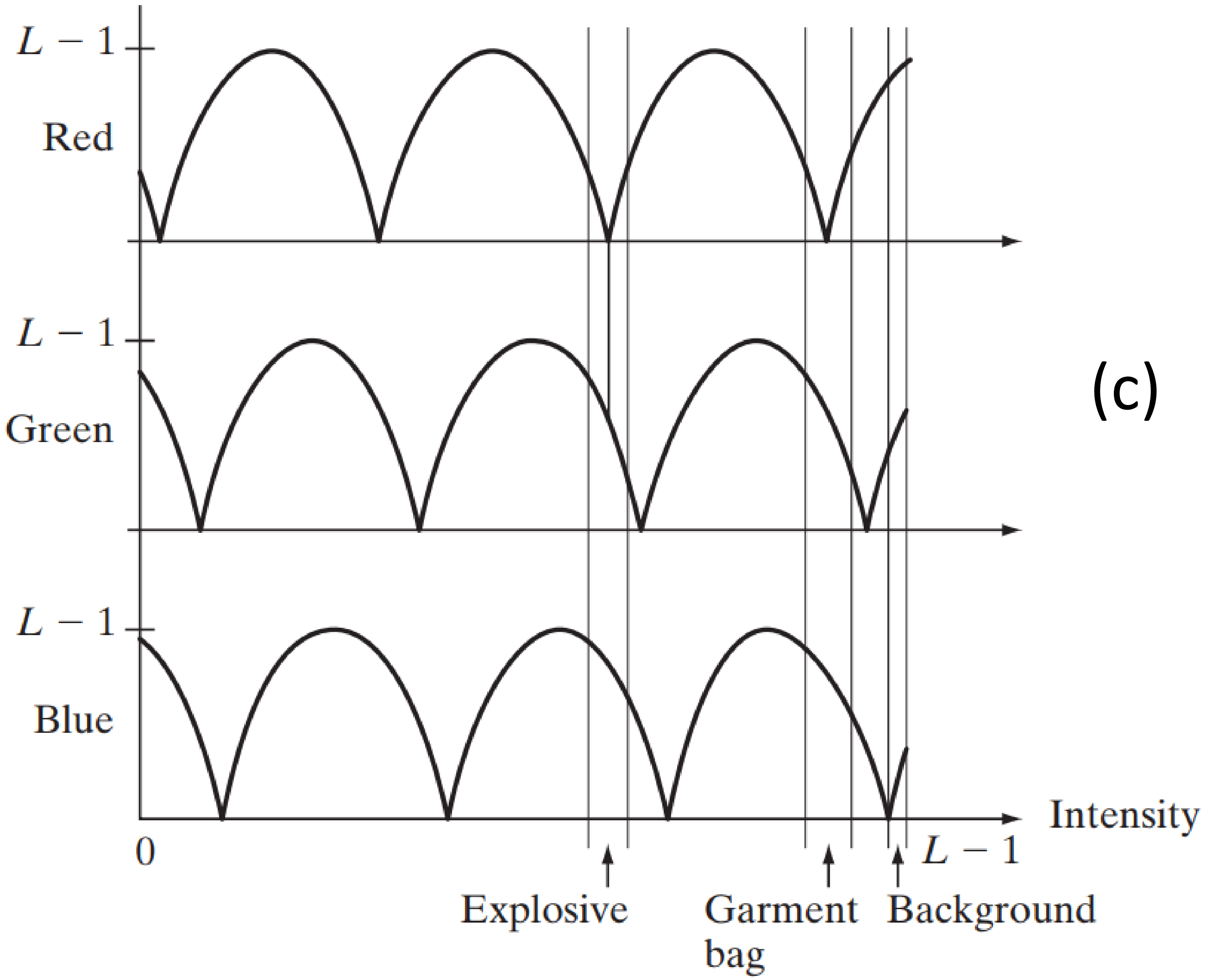
****** Pixels with intensity values in the steep section* of the sinusoids are *assigned a much stronger color content* as a result of significant differences between the amplitudes of the three sinusoids caused by the phase displacement between them.



(b)

The image shown in Fig. (c) was obtained with the transformation functions in Fig. (b), which shows the gray-level bands corresponding to the explosive, garment bag, and background, respectively.

Note that the explosive and background have quite different intensity levels, but they were both *coded with approximately the same color as a result of the periodicity of the sine waves*.

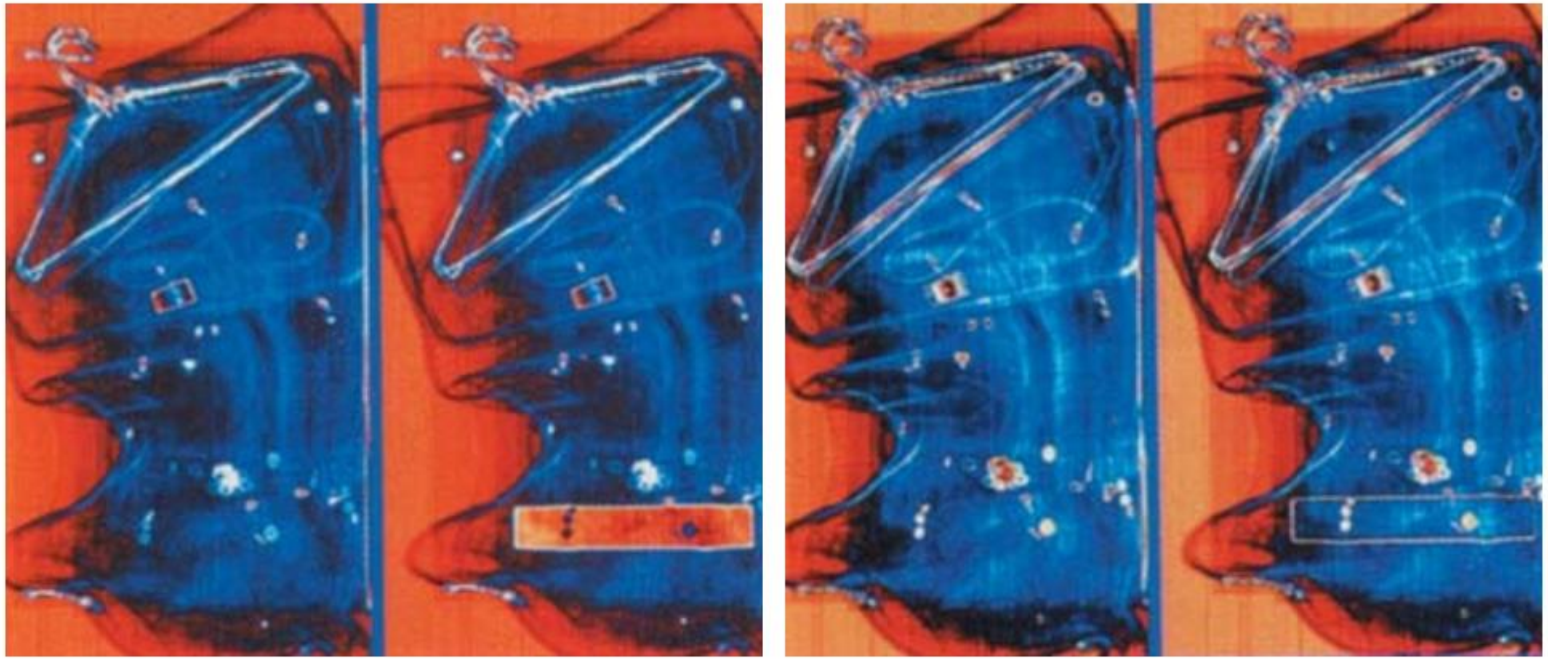


(c)

The image shown in Fig. (d) was obtained with the transformation functions in Fig. (c). In this case the explosives and garment bag intensity bands were mapped by similar transformations and thus received essentially the same color assignments.

Note that this mapping allows an observer to “see” through the explosives.

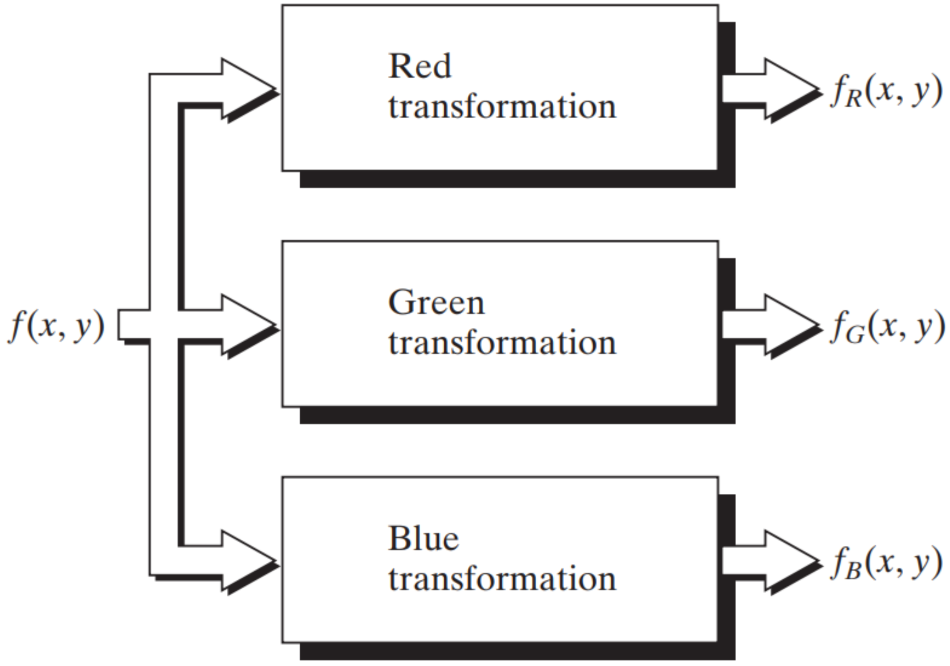
The background mappings were about the same as those used for Fig. (c), producing almost identical color assignments.



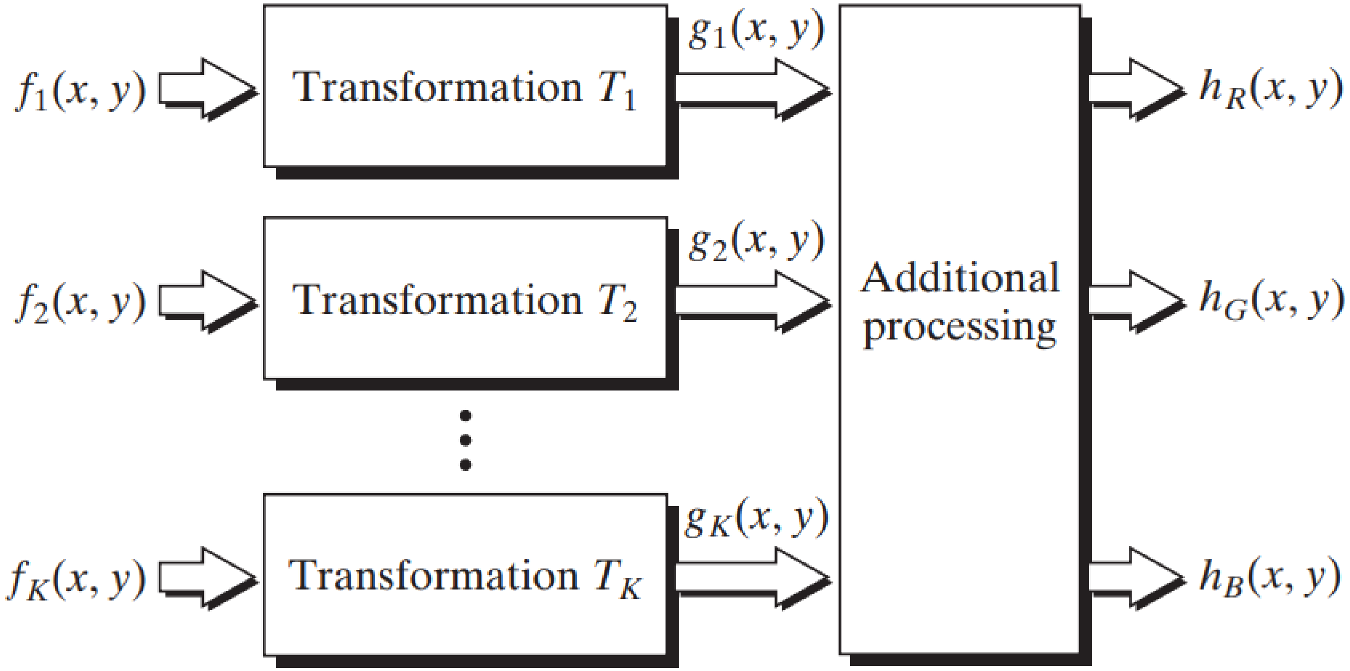
(d)

The approach shown in Fig. (f) is based on a single monochrome image.

Often, it is of interest to *combine several monochrome images into a single color composite*, as shown in Fig. (g).

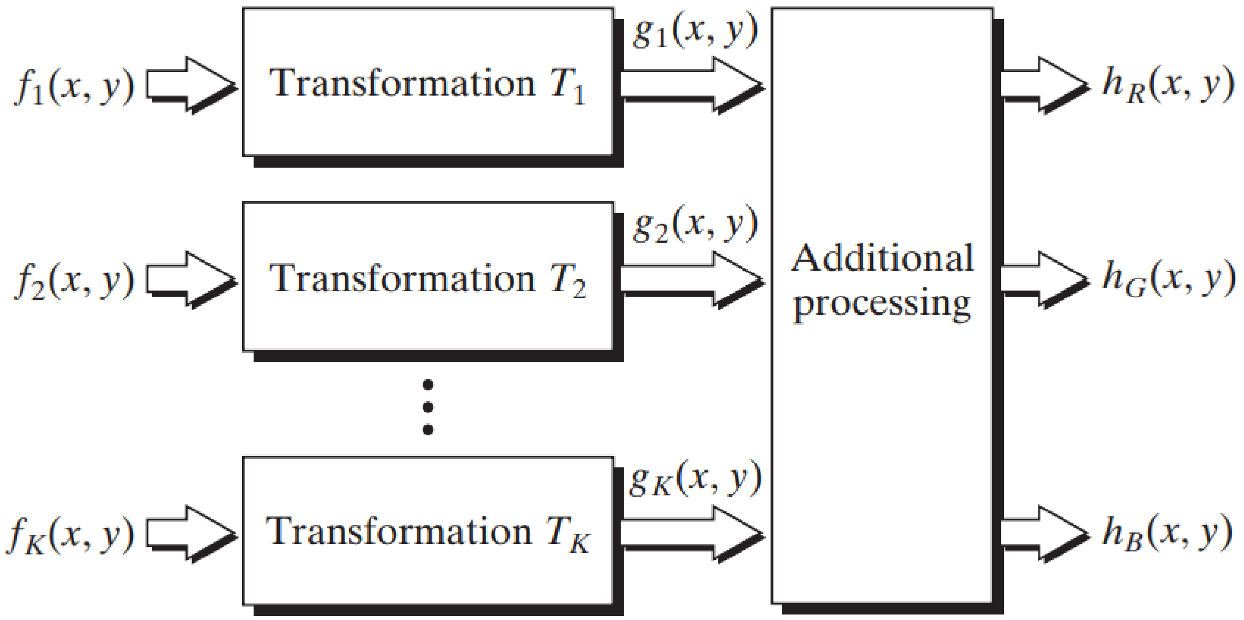


(f)



(g)

A frequent use of this approach is in multispectral image processing, where *different sensors produce individual monochrome images*, each in a different spectral band.

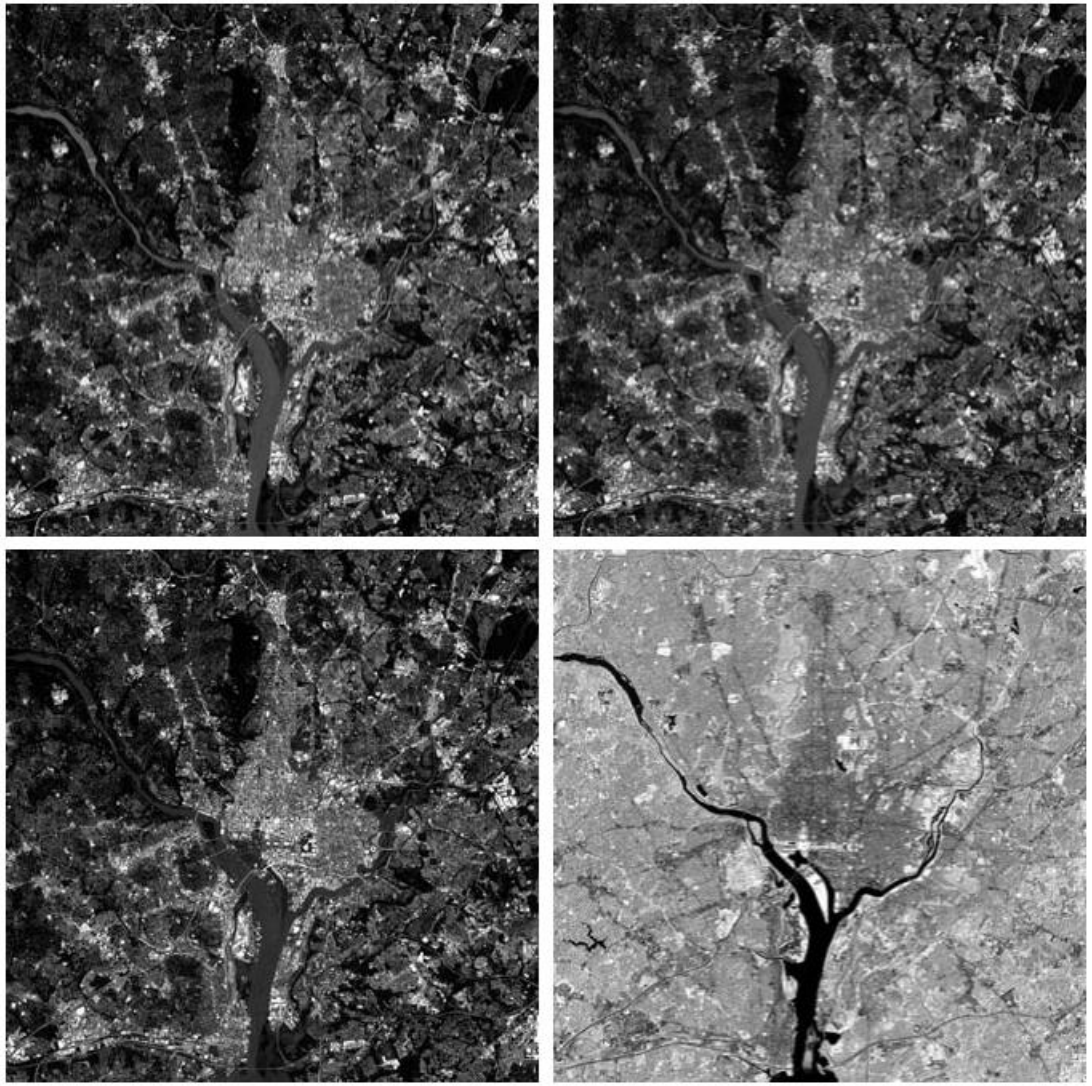


(g)

The types of additional processes shown in Fig. (g) can be techniques such as color balancing, combining images, and selecting the three images for display *based on knowledge about response characteristics of the sensors used to generate the images.*

Example: Figures (a) through (d) show four spectral satellite images of Washington, D.C., including part of the Potomac River. *The first three images are in the visible red, green, and blue*, and the *fourth is in the near infrared* (see Table (a) and Fig. (g)).

(a)–(d) Images in bands 1–4



(a) (b)

(c) (d)

Table (a) Thematic bands in NASA's LANDSAT satellite.

Band No.	Name	Wavelength (μm)	Characteristics and Uses
1	Visible blue	0.45–0.52	Maximum water penetration
2	Visible green	0.52–0.60	Good for measuring plant vigor
3	Visible red	0.63–0.69	Vegetation discrimination
4	Near infrared	0.76–0.90	Biomass and shoreline mapping
5	Middle infrared	1.55–1.75	Moisture content of soil and vegetation
6	Thermal infrared	10.4–12.5	Soil moisture; thermal mapping
7	Middle infrared	2.08–2.35	Mineral mapping

LANDSAT satellite images of the Washington, D.C. area. The numbers refer to the thematic bands in Table

(g)

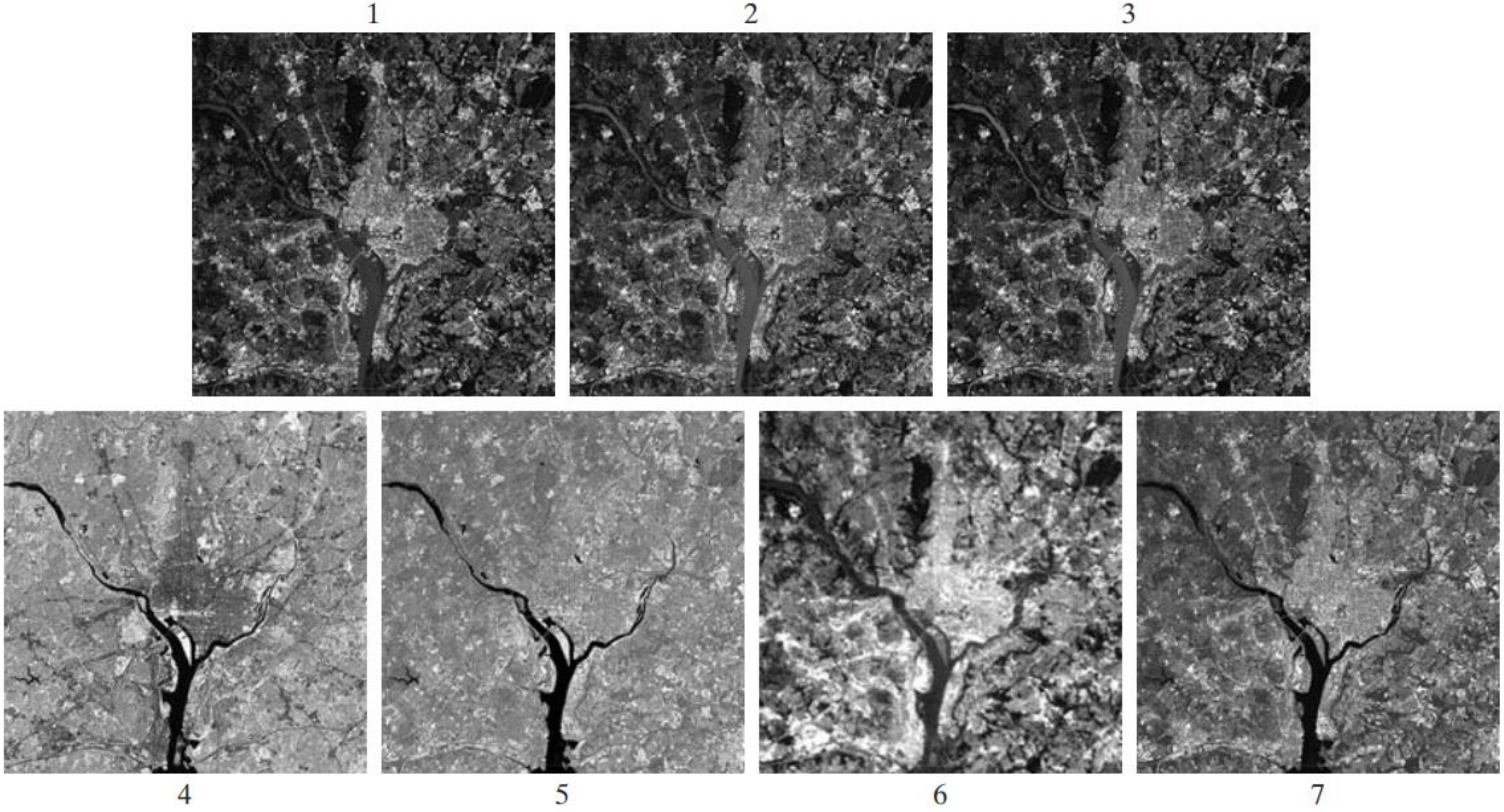


Figure (e) is the full-color image obtained by *combining the first three images into an RGB image*.

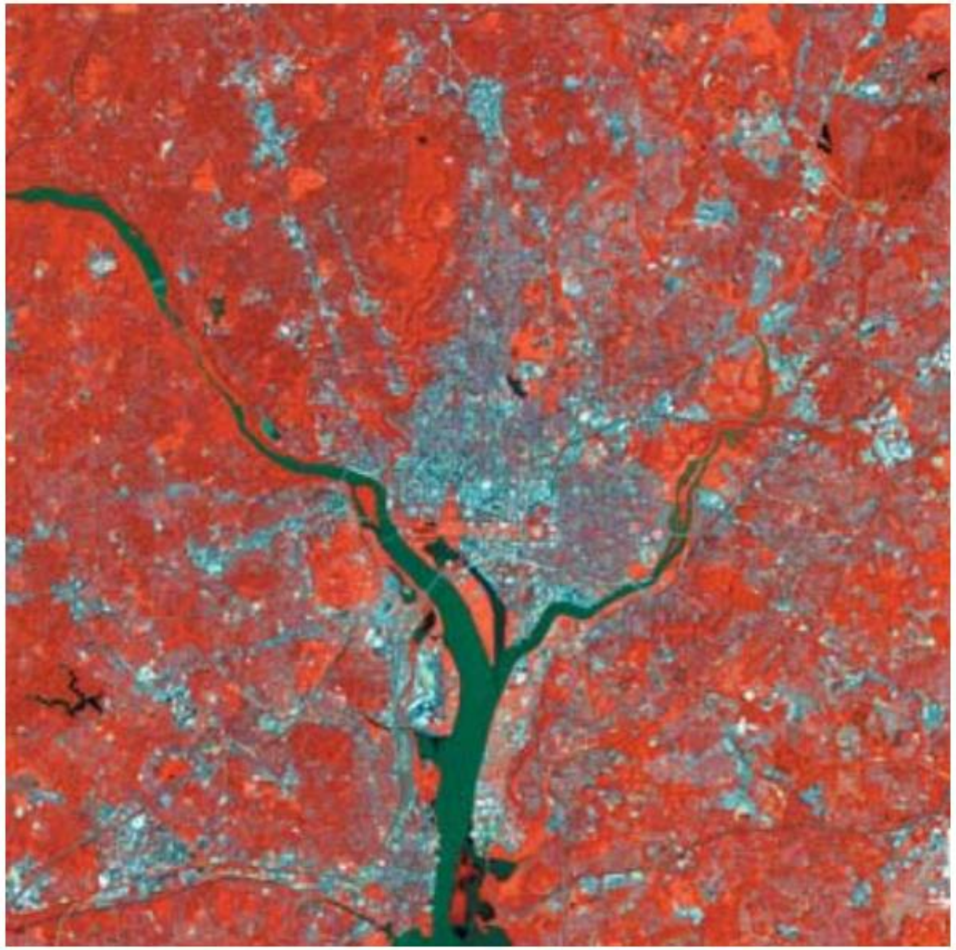
Full-color images of *dense areas are difficult to interpret*, but one **notable feature** of this image is the *difference in color in various parts of the Potomac River*.

(e) Color composite image obtained by treating (a), (b), and (c) as the red, green, blue components of an RGB image.

Figure (f) is a little more interesting. This image was formed by *replacing the red component of Fig. (e) with the near-infrared image.*

From Table (a), we know that this band is strongly responsive to the biomass components of a scene.

Figure (f) shows quite clearly the difference between *biomass (in red)* and the *human-made features* in the scene, composed primarily of concrete and asphalt, which *appear bluish in the image*.



(f) Image obtained in the same manner, but using in the red channel the near-infrared image in (d).

The type of processing just illustrated is quite powerful in helping visualize events of interest in *complex images*, especially when *those events are beyond our normal sensing capabilities*.

Example: Figure (a) is an excellent illustration of this. These are images of the Jupiter moon Io, shown in pseudocolor by combining several of the sensor images from the Galileo spacecraft, *some of which are in spectral regions not visible to the eye*.

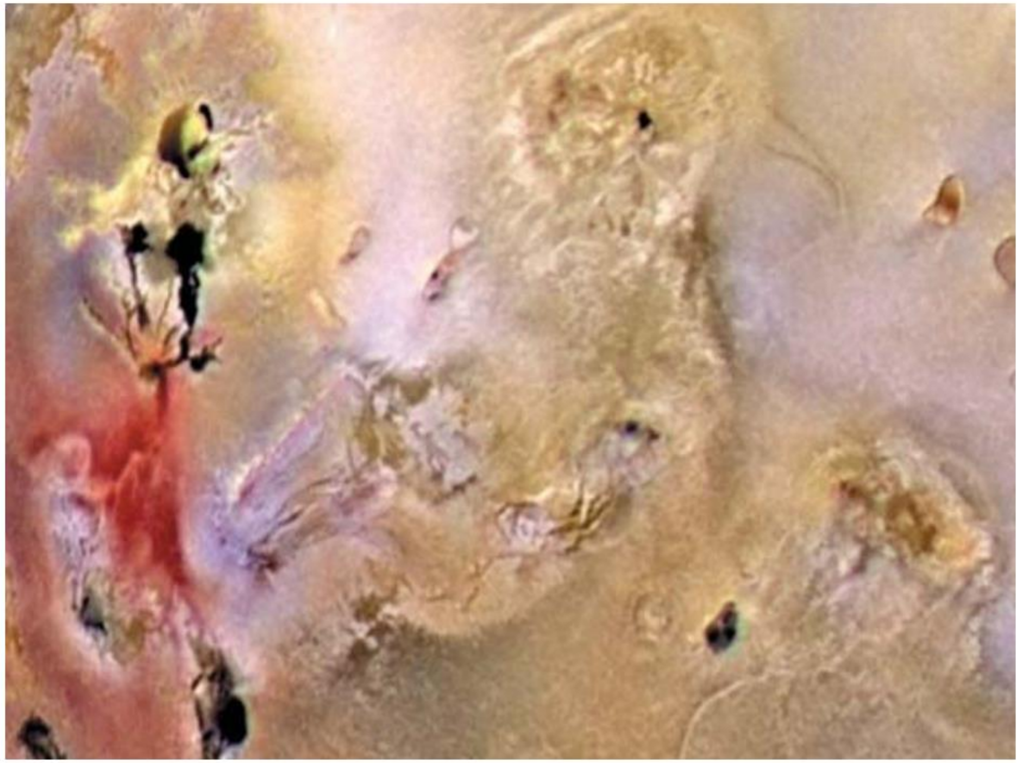
(a) Pseudocolor rendition of Jupiter Moon Io.

However, by *understanding the physical and chemical processes* likely to *affect sensor response*, it is possible to *combine the sensed images into a meaningful pseudocolor map.*

One way to *combine the sensed image* data is by *how they show either differences in surface chemical composition* or *changes in the way the surface reflects sunlight*.

(a) Pseudocolor rendition of Jupiter Moon Io.

For example, in the pseudocolor image in Fig. (b), *bright red depicts material newly ejected from an active volcano* on Io, and the surrounding *yellow materials are older sulfur deposits*. This image conveys these characteristics much more readily than would be possible by analyzing the component images individually.



(b) A close-up

2.7 Basics of Full-Color Image Processing

Full-color image processing approaches *fall into two major categories*.

- In the first category, we *process each component image individually* and then *form a composite processed color image* from the individually processed components.

- In the second category, we *work with color pixels directly*. Because full-color images *have at least three components*, *color pixels are vectors*.

For example, in the RGB system, *each color point can be interpreted* as a *vector extending from the origin to that point in the RGB coordinate system*.

Let c represent an arbitrary vector in RGB color space:

$$\mathbf{c} = \begin{bmatrix} c_R \\ c_G \\ c_B \end{bmatrix} = \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

This equation indicates that the components of c are simply the RGB components of a color image at a point.

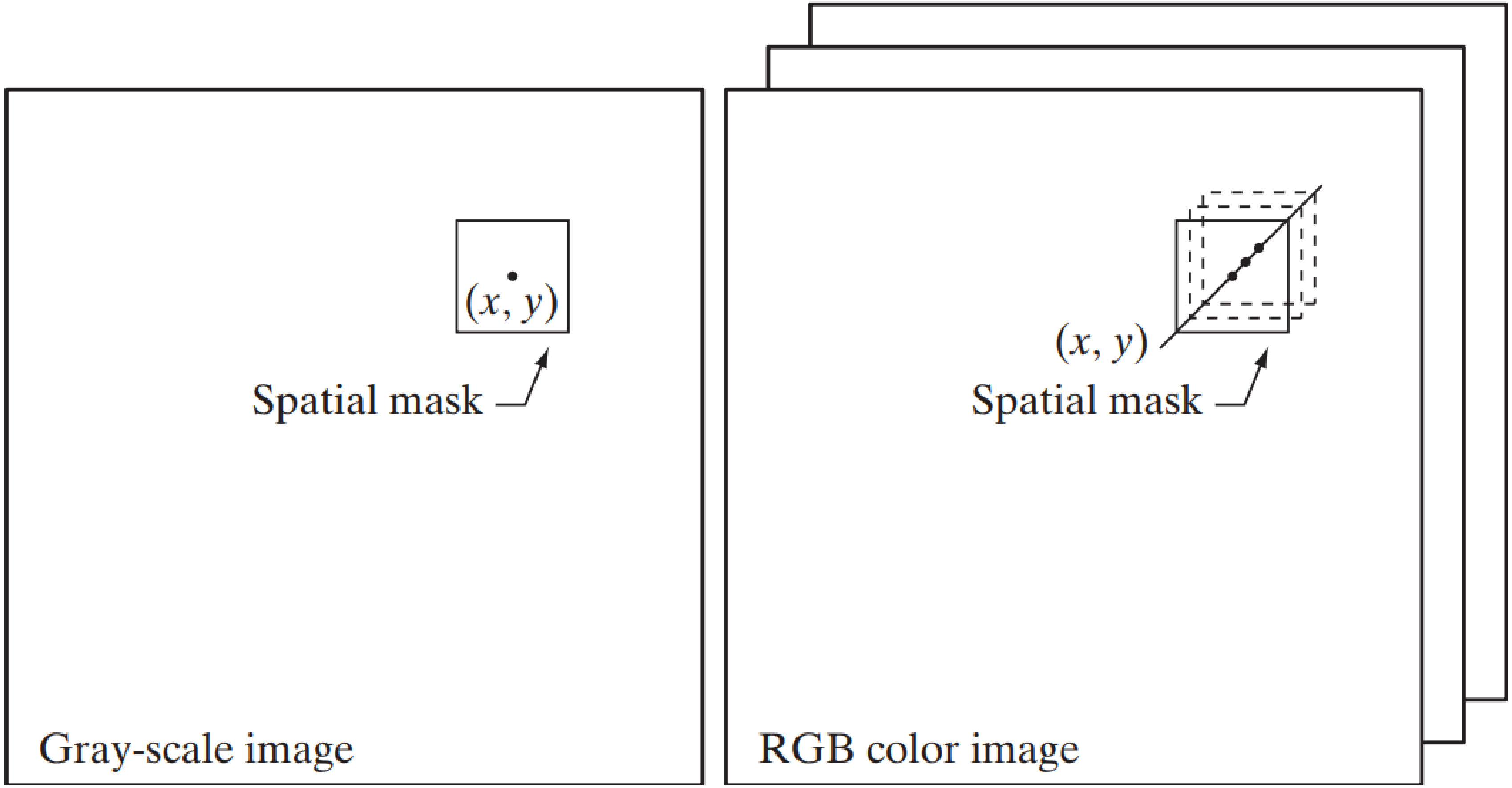
We take into account the fact that the color components are a *function of coordinates (x, y)* by using the notation

$$\mathbf{c}(x, y) = \begin{bmatrix} c_R(x, y) \\ c_G(x, y) \\ c_B(x, y) \end{bmatrix} = \begin{bmatrix} R(x, y) \\ G(x, y) \\ B(x, y) \end{bmatrix}$$

For an image of size $M \times N$, there are MN such vectors, $\mathbf{c}(x, y)$, for $x = 0, 1, 2, \ldots, M - 1$; $y = 0, 1, 2, \ldots, N - 1$.

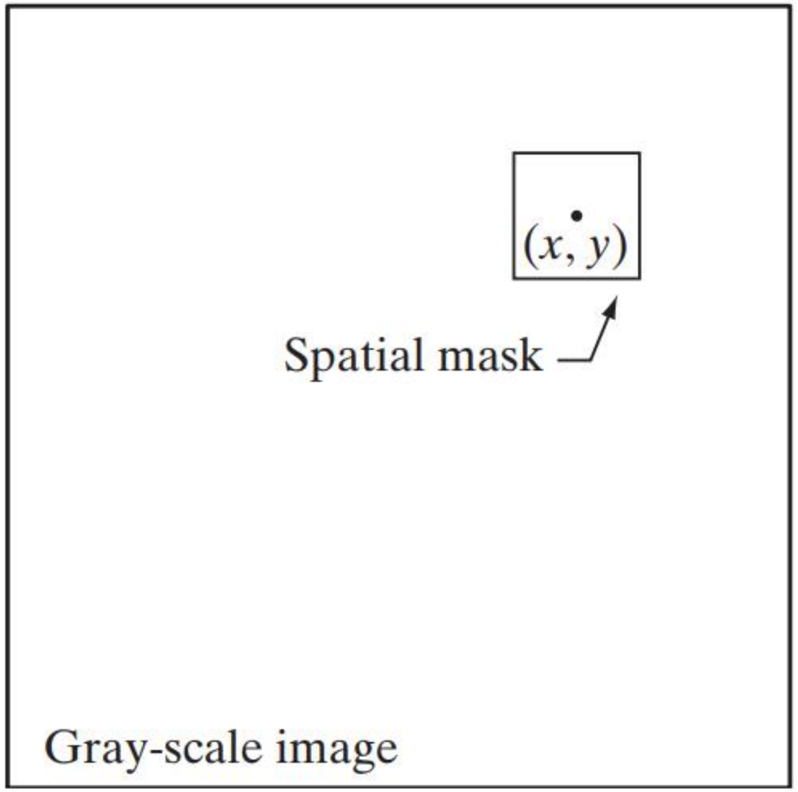
It is important to keep in mind that Eq. depicts a vector whose components are spatial variables in x and y. This is a frequent source of confusion that can be avoided by focusing on the fact that our interest lies in spatial processes. That is, we are interested in image processing techniques formulated in x and y.

As an illustration, Fig. shows neighborhood spatial processing of gray-scale and full-color images.

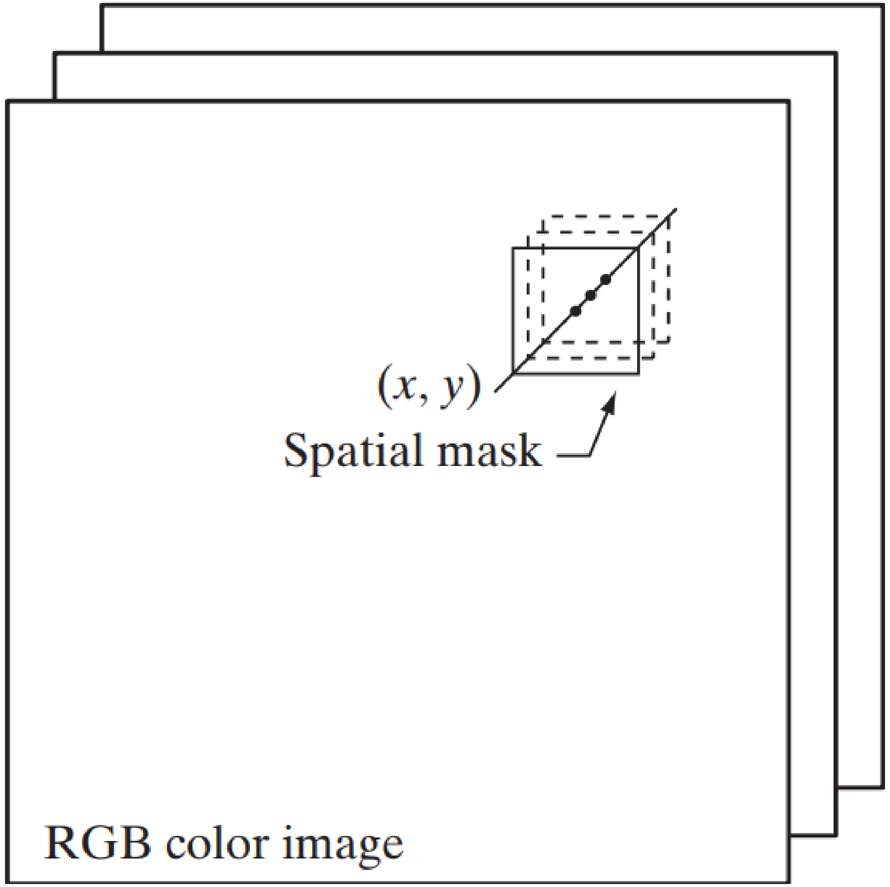


Spatial masks for gray-scale and RGB color images

Suppose that the process is neighborhood averaging. In Fig. (a), *averaging would be accomplished by summing the intensities of all the pixels in the neighborhood and dividing by the total number of pixels* in the neighborhood.



In Fig. (b), averaging would be done by summing all the vectors in the neighborhood and *dividing each component by the total number of vectors* in the neighborhood.



2.7.1 Color Transformations

The techniques described in this section, collectively called color transformations, deal with processing the components of a color image within the context of a single color model, as opposed to the conversion of those components between models.

model color transformations using the expression

$$g(x, y) = T[f(x, y)]$$

where f(x,y) is a color input image, g(x,y) is the transformed or processed color output image, and T is an operator on f over a spatial neighborhood of (x, y).

Analogous to the approach we used to introduce the basic intensity transformations, we will restrict attention in this section to color transformations of the form

$$s_i = T_i(r_1, r_2, \ldots, r_n), \qquad i = 1, 2, \ldots, n$$

where, for notational simplicity, r_i and s_i are variables denoting the color components of f(x,y) and g(x,y) at any point (x, y), n is the number of color components, and $\{T_1, T_2, \ldots, T_n\}$ is a set of transformation or color mapping functions that operate on r_i to produce s_i.

2.7.2 Color Image Smoothing

$$\bar{\mathbf{c}}(x, y) = \begin{bmatrix} \dfrac{1}{K} \displaystyle\sum_{(s,t)\in S_{xy}} R(s,t) \\ \dfrac{1}{K} \displaystyle\sum_{(s,t)\in S_{xy}} G(s,t) \\ \dfrac{1}{K} \displaystyle\sum_{(s,t)\in S_{xy}} B(s,t) \end{bmatrix}$$

2.7.3 Color Image Sharpening

$$\nabla^2[\mathbf{c}(x, y)] = \begin{bmatrix} \nabla^2 R(x, y) \\ \nabla^2 G(x, y) \\ \nabla^2 B(x, y) \end{bmatrix}$$